Bhoomi Sakhrani
2401097
AI&DS-T22
ASD&D

Experiment 3

LO: LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory

1. Introduction to Docker Compose

Docker Compose is a powerful orchestration tool designed to simplify the definition and management of multi-container Docker applications. In a typical containerized environment, complex applications rarely rely on a single service. Instead, they often require a web server, a database, a cache, and other background workers running simultaneously. Manually starting and linking these containers using individual *docker run* commands is cumbersome and error-prone. Docker Compose solves this by allowing developers to define an entire application stack in a single, human-readable YAML configuration file (*docker-compose.yml*).

2. Core Architecture and Concepts

Docker Compose operates based on three key pillars:

- **Services:** These are the building blocks of your application. Each service defines a container that runs a specific image. You can configure environment variables, ports, volume mounts, and network dependencies for each service individually.
- **Networks:** Docker Compose automatically creates a default bridge network for your application. This allows containers defined within the same *docker-compose.yml* file to communicate with each other using service names as hostnames, effectively abstracting the complexity of inter-service networking.
- **Volumes:** Essential for data persistence, volumes allow you to map directories from your host machine or named Docker volumes to the container's file system, ensuring that data is not lost when containers are stopped or removed.

3. Key Advantages

- **Simplified Environment Setup:** With a single command (*docker compose up*), you can spin up an entire environment, including all dependencies, databases, and services, drastically reducing onboarding time for new developers.
- **Consistency:** By defining the infrastructure as code (IaC) via YAML, you ensure that the development, testing, and production environments are identical, eliminating the classic "it works on my machine" problem.

Bhoomi Sakhrani
2401097
AI&DS-T22
ASD&D

- **Scalability:** Docker Compose makes it easy to scale individual services horizontally by simply updating the *deploy* configuration or using flags, allowing the application to handle increased load efficiently.
- **Isolation:** Applications defined in one Compose project are isolated from others by default, preventing conflicts between dependencies or port mappings.

4. Common Use Cases

- **Microservices Architectures:** Managing the development and integration testing of multiple interconnected services.
- **Automated Testing:** Creating clean, isolated environments for running test suites against database and service dependencies.
- **Local Development:** Emulating production-like infrastructure locally for rapid iteration and debugging.

5. Essential Docker Compose Commands

Managing the lifecycle of your application stack is straightforward with the *docker compose* CLI. The most frequently used commands are:

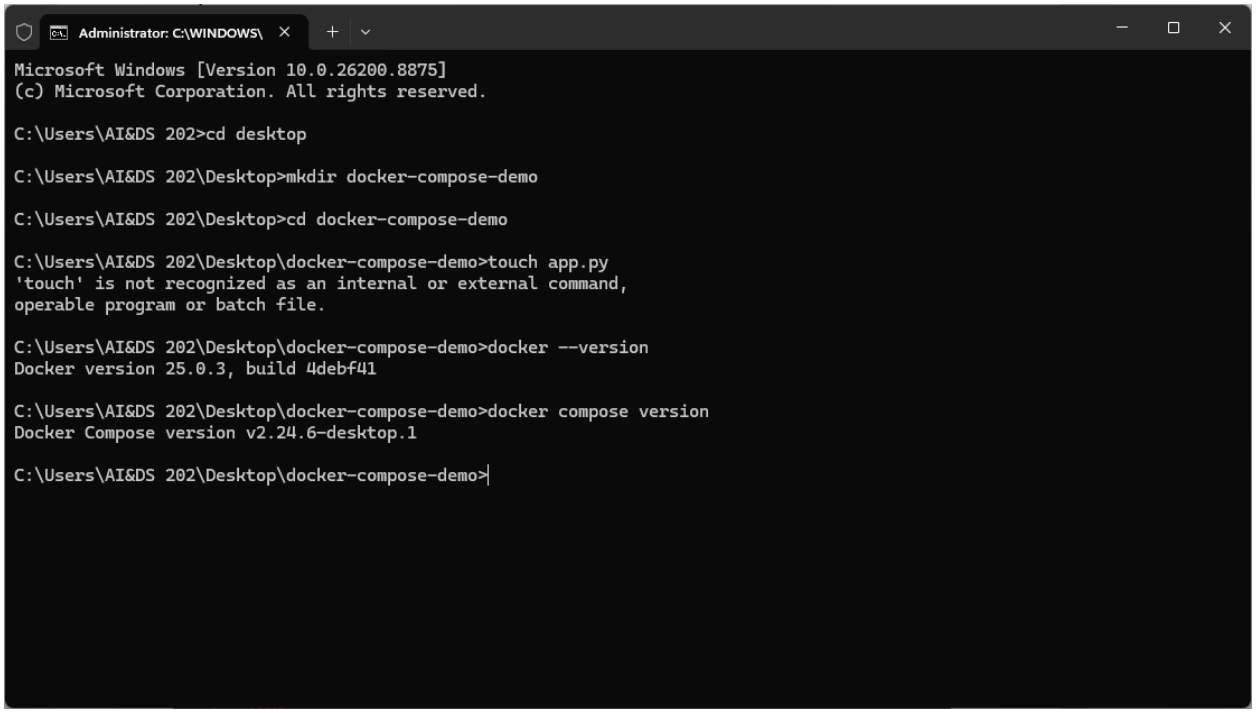
Command	**Description**
`docker compose up`	Reads the *docker-compose.yml* file, pulls images if necessary, creates containers, networks, and volumes, and starts the services. Using the --*build* flag forces a rebuild of images before starting.
`docker compose down`	Stops containers and removes them, along with the networks created by *docker compose up*. It cleans up the environment completely.
`docker compose ps`	Lists all containers associated with the project and displays their status (Up/Exit), ports, and names.
`docker compose logs`	Retrieves and displays the combined log output from all services, which is invaluable for debugging startup issues.
`docker compose restart`	Restarts all running containers associated with the project without recreating them.

Command	Description
`docker compose stop`	Stops the running containers without removing them, preserving the state of the containers.

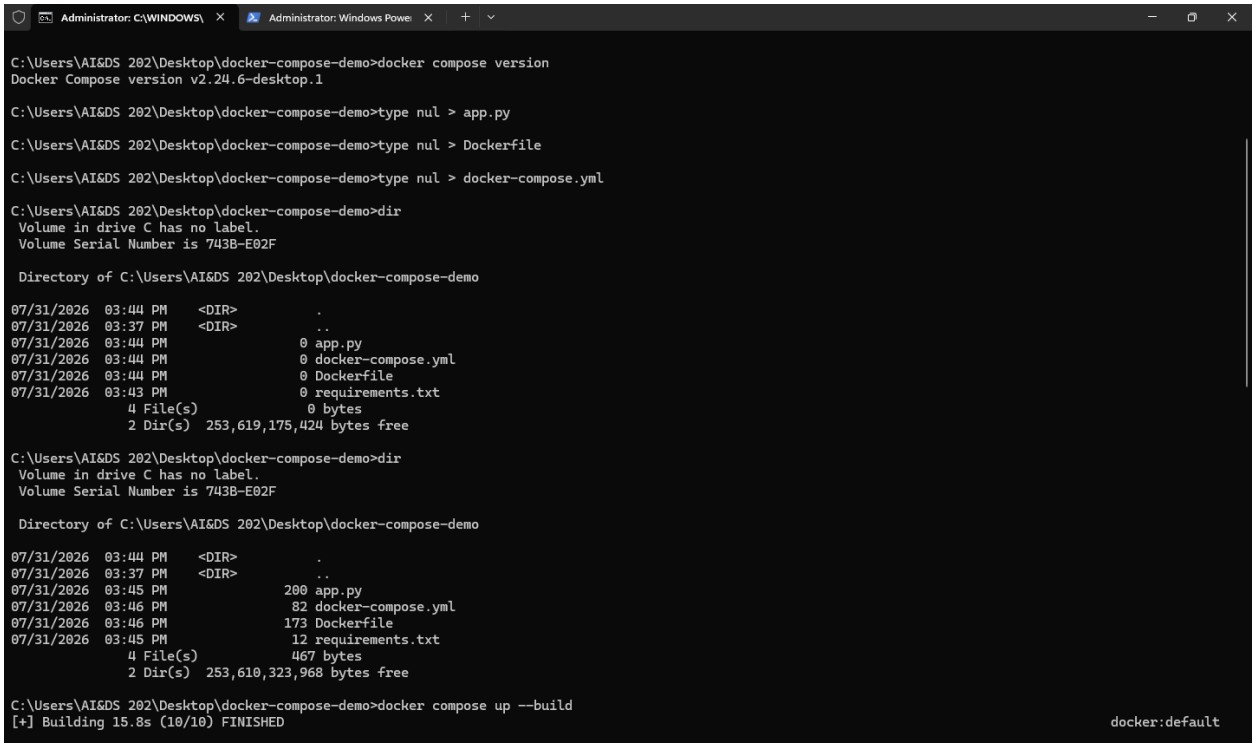
6. Workflow Summary

The standard workflow for a Docker Compose project involves defining the application services in the *docker-compose.yml*, building the necessary Dockerfiles for custom services, and then executing the orchestration commands to bring the stack online. This approach not only streamlines deployment but also enforces best practices in containerization, making infrastructure management predictable and automated.

Result:



Bhoomi Sakhrani
2401097
AI&DS-T22
ASD&D



```
Administrator: C:\WINDOWS\ X  Administrator: Windows Powe X  +  v

C:\Users\AI&DS 202\Desktop\docker-compose-demo>docker compose version
Docker Compose version v2.24.6-desktop.1

C:\Users\AI&DS 202\Desktop\docker-compose-demo>type nul > app.py

C:\Users\AI&DS 202\Desktop\docker-compose-demo>type nul > Dockerfile

C:\Users\AI&DS 202\Desktop\docker-compose-demo>type nul > docker-compose.yml

C:\Users\AI&DS 202\Desktop\docker-compose-demo>dir
 Volume in drive C has no label.
 Volume Serial Number is 743B-E02F

 Directory of C:\Users\AI&DS 202\Desktop\docker-compose-demo

07/31/2026  03:44 PM    <DIR>          .
07/31/2026  03:37 PM    <DIR>          ..
07/31/2026  03:44 PM                 0 app.py
07/31/2026  03:44 PM                 0 docker-compose.yml
07/31/2026  03:44 PM                 0 Dockerfile
07/31/2026  03:43 PM                 0 requirements.txt
               4 File(s)              0 bytes
               2 Dir(s)  253,619,175,424 bytes free

C:\Users\AI&DS 202\Desktop\docker-compose-demo>dir
 Volume in drive C has no label.
 Volume Serial Number is 743B-E02F

 Directory of C:\Users\AI&DS 202\Desktop\docker-compose-demo

07/31/2026  03:44 PM    <DIR>          .
07/31/2026  03:37 PM    <DIR>          ..
07/31/2026  03:45 PM               200 app.py
07/31/2026  03:46 PM                82 docker-compose.yml
07/31/2026  03:46 PM               173 Dockerfile
07/31/2026  03:45 PM                12 requirements.txt
               4 File(s)            467 bytes
               2 Dir(s)  253,610,323,968 bytes free

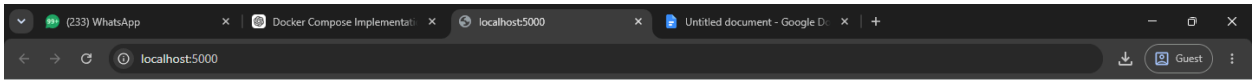
C:\Users\AI&DS 202\Desktop\docker-compose-demo>docker compose up --build
[+] Building 15.8s (10/10) FINISHED                                                
```

Bhoomi Sakhrani
2401097
AI&DS-T22
ASD&D

```
web-1  | Press CTRL+C to quit
web-1  | 172.18.0.1 - - [31/Jul/2026 10:17:41] "GET / HTTP/1.1" 200 -
web-1  | 172.18.0.1 - - [31/Jul/2026 10:17:41] "GET /favicon.ico HTTP/1.1" 404 -
Gracefully stopping... (press Ctrl+C again to force)
[+] Stopping 1/1
 ✔Container docker-compose-demo-web-1  Stopped                                  10.4s
canceled

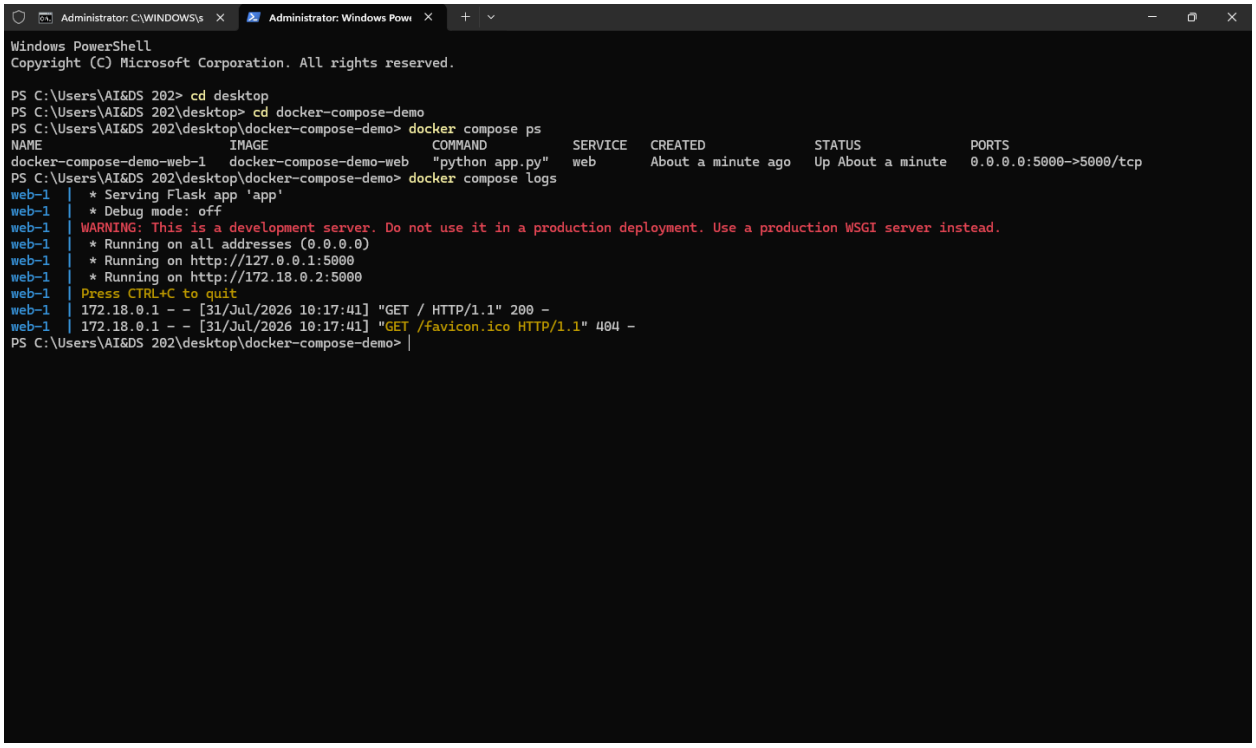
C:\Users\AI&DS 202\Desktop\docker-compose-demo>docker compose restart
[+] Restarting 1/1
 ✔Container docker-compose-demo-web-1  Started                                  0.4s

C:\Users\AI&DS 202\Desktop\docker-compose-demo>docker compose down
[+] Running 2/2
 ✔Container docker-compose-demo-web-1  Removed                                  10.4s
 ✔Network docker-compose-demo_default  Removed                                  0.2s
```



Docker Compose is working!

Bhoomi Sakhrani
2401097
AI&DS-T22
ASD&D



Conclusion: This experiment successfully demonstrated the use of Docker Compose to manage and deploy a multi-service application, highlighting its efficiency in environment consistency and service orchestration.